University of Global Village (UGV), Barisal

PROJECT-BASED LEARNING PROPOSAL

Shopping Mall Sales Management System

A Single-Shop Java Swing Desktop POS & Inventory Application

JAVA · MYSQL · JDBC · POS BILLING

Submitted By

Name	ID
Chandan Karmaker	12511006
Emon Hossain	12511043

SUPERVISOR

Md. Riadul Islam
Department Head, Assistant Professor

Signature

Shopping Mall Sales Management System

A Single-Shop Java Swing Desktop POS & Inventory Application

01 Executive Summary

Small shop counters often rely on manual registers or spreadsheets for billing and stock tracking. This proposal describes a Shopping Mall Sales Management System (SMSMS) — a Java Swing desktop application backed by a MySQL database — that lets a single shop record product sales, keep stock counts accurate, and review a simple sales history, all from one role-secured application.

In one line: a simple, offline desktop POS and inventory tool that replaces manual billing and stock tracking for a single shop.

02 Background & Motivation

Many small shops still bill customers by hand or with basic spreadsheets, with no automatic link between a sale and the stock it should reduce. This leads to stock counts that drift from reality and no easy way to see how much was sold on a given day.

This project is motivated by the gap between expensive commercial POS software and what a beginner-friendly, learning-focused project can realistically deliver in one semester: a small but complete, correctly-built system rather than a large, half-finished one.

03 Problem Statement

- Manual billing: sales are written down or tracked in a spreadsheet with no automatic stock update.
- Manual inventory: stock counts drift because sales and stock aren't linked.
- No shared login/roles: anyone can access all functions of whatever tool is currently used.
- No quick sales overview: no simple way to see today's total sales at a glance.

04 Aim & Objectives

Aim: To design and implement a simple, role-based desktop application that lets a single shop record sales and manage product inventory using Java Swing and MySQL.

Specific Objectives

- Design a small, normalized MySQL schema for products, sales, and sale items.
- Implement a POS billing screen that always updates stock correctly using a JDBC transaction.
- Implement basic role-based access for Admin and Cashier.
- Provide a simple sales history screen and a "today's total" figure.
- Package the system as a runnable desktop application requiring no internet dependency.

05 Scope of the Project

In Scope

- Product management (add, edit, delete, view).

- POS billing: cart, one tax rate, one bill-level discount, payment mode selection, receipt.
- Automatic, transaction-safe stock reduction on sale.
- Sales history with a simple daily total.
- Admin and Cashier login with role-based screens.

Out of Scope (Future Work)

- Multi-shop / multi-branch support.
- Barcode scanning and label printing.
- Customer records, loyalty points, and coupons.
- Supplier and purchase-order tracking.
- Employee attendance tracking.
- Returns and refunds.
- Charts, dashboards, and PDF/Excel export.
- Cloud backup or remote access.

06 Development Methodology

The project will be built incrementally: each milestone adds one working, end-to-end feature (e.g., "products can be added and listed") rather than building all screens partially at once. This keeps a demoable version available from an early stage.

07 Proposed System Architecture

Layer	Responsibility
Presentation (Swing)	Login, Products, POS, Sales History, and User screens; input validation
DAO	All SQL via PreparedStatement (ProductDAO, SaleDAO, UserDAO); business checks like stock validation live in the form's event-handler methods
MySQL Database	Single-shop schema: users, products, sales, sale_items

This is a 3-layer version of the original 5-layer MVC design (Presentation / Controller / Service / DAO / Database). Splitting out Controller and Service layers is a good next step once the simplified version works.

08 Data Flow Diagram

The Level 1 Data Flow Diagram below shows how information moves between the two user roles, the five core processes, and the four data stores in the simplified system.

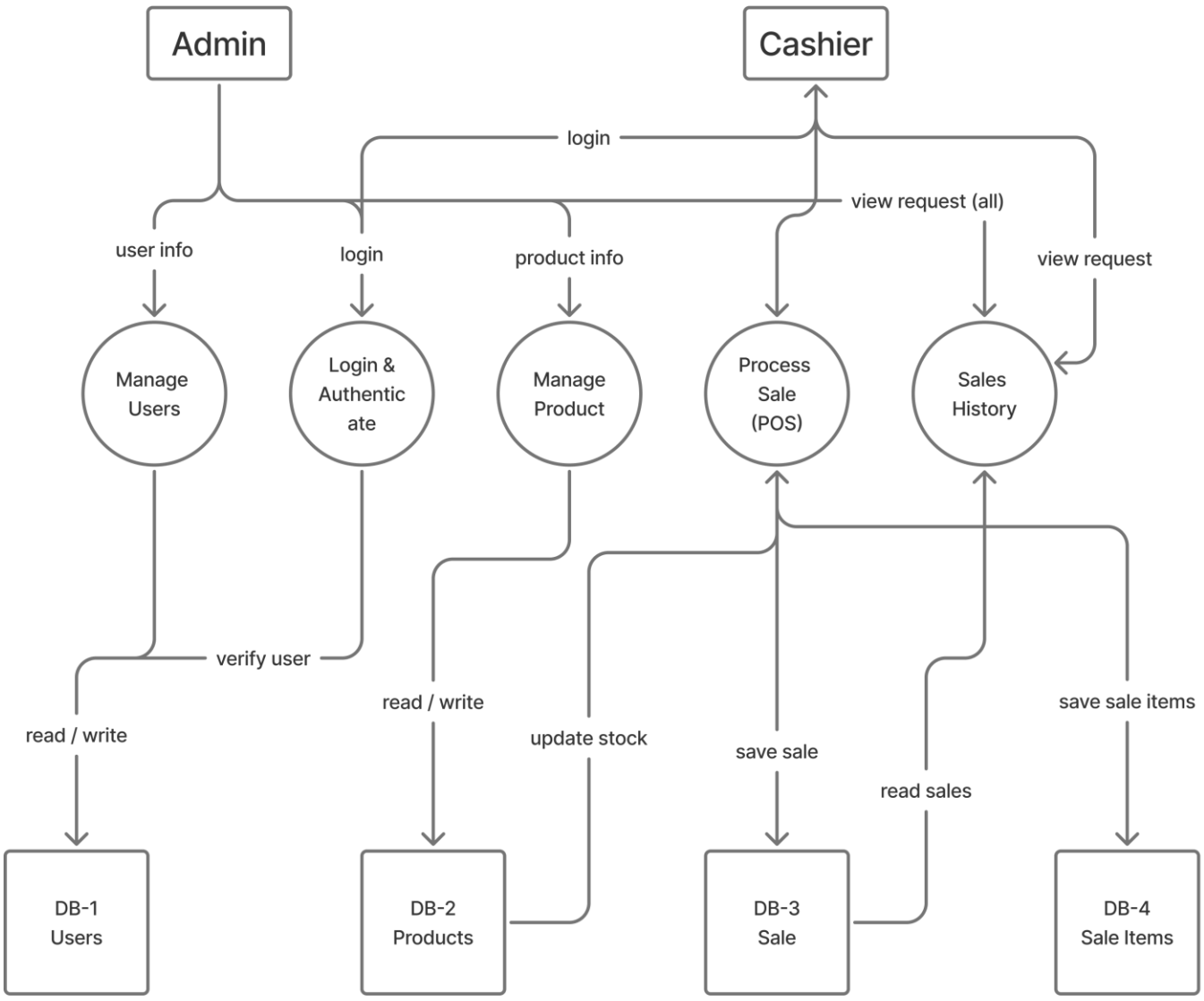


Figure 1 — Level 1 Data Flow Diagram

9 UML Class Diagram

The class diagram below shows the four core domain classes used by the simplified system, their key attributes and methods, and how they relate to one another.

UML Class Diagram

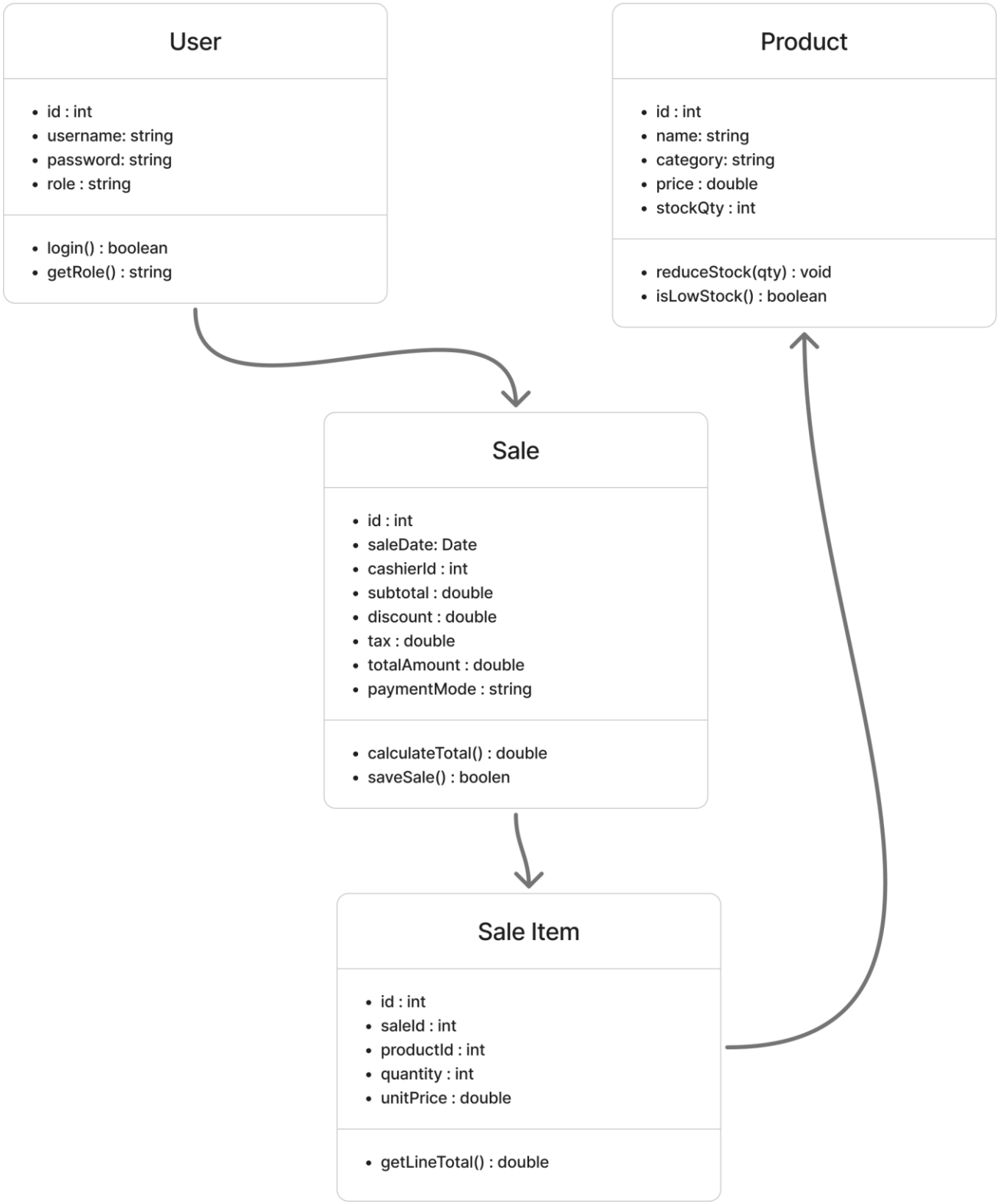


Figure 2 — UML Class Diagram (Core Domain Model)

- A User (cashier) can process many Sales, but each Sale is processed by exactly one User.
- A Sale is composed of one or more SaleItem records — if a Sale is deleted, its SaleItem rows go with it (composition).
- Each SaleItem refers to exactly one Product, but a Product can appear in many SaleItem records over time.

10 Database Structure

The simplified schema has four tables: one for logins, one for the product catalog, and two that record a completed sale and its line items. All money columns use DECIMAL(10,2) and all tables use the InnoDB engine so multi-step operations (like completing a sale) can be committed or rolled back as one unit.

users

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user ID
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(255)	NOT NULL	Login password
role	VARCHAR(20)	NOT NULL	'Admin' or 'Cashier'

products

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product ID
name	VARCHAR(100)	NOT NULL	Product name
category	VARCHAR(50)	—	Simple plain-text category label
price	DECIMAL(10,2)	NOT NULL	Selling price per unit
stock_qty	INT	NOT NULL, DEFAULT 0	Current stock on hand

sales

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique sale / invoice ID
sale_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time of the sale
cashier_id	INT	FOREIGN KEY → users(id), NOT NULL	Cashier who processed the sale

subtotal	DECIMAL(10,2)	NOT NULL	Sum of line items before discount/tax
discount	DECIMAL(10,2)	DEFAULT 0	Bill-level discount applied
tax	DECIMAL(10,2)	DEFAULT 0	Tax amount applied
total_amount	DECIMAL(10,2)	NOT NULL	Final payable amount
payment_mode	VARCHAR(20)	NOT NULL	'Cash' or 'Card'

sale_items

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique line-item ID
sale_id	INT	FOREIGN KEY → sales(id), NOT NULL	Parent sale/invoice
product_id	INT	FOREIGN KEY → products(id), NOT NULL	Product sold
quantity	INT	NOT NULL	Quantity sold
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit at time of sale (copied from product, so later price changes don't rewrite history)

11 Proposed Timeline

Estimated over an 8-week development cycle:

Phase	Duration
Requirements & DB Design	Week 1
Core Setup & Login	Week 1 – 2
Product Management	Week 2 – 3
POS / Billing Module	Week 3 – 5
Sales History & User Management	Week 5 – 6
Testing & Polish	Week 6 – 7
Documentation & Defense Prep	Week 7 – 8

12 Tools & Technologies

Java 17+ · Java Swing · MySQL 8 · JDBC (MySQL Connector)

13 Feasibility Study

Aspect	Assessment
Technical	Uses only core Java, Swing, and JDBC — all well-documented and appropriate for a beginner.
Economic	Fully open-source toolchain; zero licensing cost.
Operational	Runs locally with no internet dependency.
Schedule	Reduced scope comfortably fits an 8-week timeline, leaving buffer for testing and writing the report.

14 Expected Outcomes

- A working, installable desktop POS and inventory application for a single shop.
- A small, correctly normalized MySQL schema with an ER diagram.
- A demo showing a full sale: search product → add to cart → complete sale → stock updates → receipt shown.
- A written report covering design decisions and testing, suitable for academic submission.

15 Future Scope

- Multi-shop support.
- Barcode scanning.
- Charts/dashboard (JFreeChart).
- PDF/Excel export.
- Returns/refunds, customer loyalty, supplier tracking, employee attendance, and an audit log.
- Password hashing and a connection pool for production-readiness.

16 Conclusion

This revised proposal keeps the core, defensible idea of the original — a transaction-safe POS and inventory system with real-time stock accuracy — while sizing the feature list to what a beginner in Java and Swing can build, test, and fully understand within a single term. It is intentionally left room to grow: every feature removed here is listed as a clear, achievable next step.

“Start with a system that correctly sells one product to one customer — everything else is an extension of that.”